

Python Data Science Stack

Core Principles & Best Practices

Vojtěch Ondryhal, K209, FVT

Analýza informačních zdrojů

LS 2026

Ekosystém: Více než součet částí

Data Science v Pythonu: Přemostění vysokoúrovňové syntaxe a C-výkonu.

- **NumPy & SciPy (Základ):**
Nízkoúrovňová pole a matematické jádro.
- **Pandas (Analytika):**
Tabulky, metadata a *Data Alignment*.
- **Vizualizace (Matplotlib/Seaborn):**
Komunikační vrstva a EDA.
- **Scikit-learn (Modelování):**
Standard pro strojové učení (ML).

Hierarchie Python Stacku

ML (Scikit-learn)
Vizuál (Seaborn)
Analýza (Pandas)
Výkon (NumPy)

*Data tečou zdola nahoru:
Od surových polí k modelům.*

Best Practice

Efektivní kód využívá **vektorizaci**. Pokud nad **tabulkou** (DataFrame) píšete for **cyklus**, pravděpodobně bojujete s nástrojem, místo abyste ho využili.

NumPy: Výkon a multidimenzionální pole

Klíčová vlastnost: Homogenní datové pole *ndarray* a *vektorizace*.

- ❑ **C-Level Performance:** Operace probíhají v kompilovaném kódu, nikoliv v interpretovaném Pythonu.
- ❑ **Broadcasting:** Inteligentní práce s poli různých tvarů bez explicitního kopírování dat.

Problém Pythonu (Overhead):

- ❑ Každý prvek je plnohodnotný objekt (metadata, reference count).
- ❑ CPU musí neustále skákat v paměti (*Cache Misses*).

Řešení NumPy (C-Backend):

- ❑ Homogenní data těsně za sebou.
- ❑ NumPy fixuje datový typ (např. `int64`). Eliminuje *Type Checking* u každého prvku.
- ❑ CPU využívá **SIMD** (*Single Instruction, Multiple Data*) – operace nad celými bloky naráz.

NumPy: Vektorizace & Memory Layout

Vektorizace vs Cyklus

```
1 import numpy as np
2
3 # Rychlá vektorizovaná operace namísto for-cyklu
4 data = np.array([10, 20, 30, 40])
5 # Proveďte se na úrovni C pro celé pole naráz
6 result = data * 2
7
8 # Agregace s využitím n-rozměrné struktury
9 matrix = np.random.rand(1000, 1000)
10 row_means = matrix.mean(axis=1)
```

Důsledek pro praxi

Místo interpretování for cyklu v Pythonu se zavolá jedna instrukce procesoru nad celým blokem paměti.
Vektorizovaný kód v NumPy bývá **50–100× rychlejší** než čistý Python.

[w-stack-vectorization.ipynb](#)

Rozložení v paměti

Python List je pole pointerů. NumPy Array je souvislý blok v RAM.

[w-stack-memory.ipynb](#)

Python List (Rozptýlené objekty)

[ptr0] →	int: 10
[ptr1] →	int: 20
[ptr2] →	int: 30
[ptr3] →	int: 40

vs.

NumPy Array (Souvislý blok)

10	20	30	40
----	----	----	----

Typ: int64, Velikost: 4 × 8 bytes

Numpy: Broadcasting

Operace mezi poli různých (ale kompatibilních) rozměrů bez kopírování dat.

□ Mechanismus:

- NumPy virtuálně „roztáhne“ menší pole (např. skalár), aby odpovídalo rozměrům většího pole.
- Nedochozí k reálné duplikaci dat v RAM.

□ Efektivita:

- Operace probíhají v **C-cyklech**.
- Skalární operace ($\times 10$) jsou typickým příkladem, ale funguje to i pro matice a vektory (Odpovídá pravidelům lineární algebry).

w-stack-broadcasting.ipynb

Vizualizace (Skalár)

Skalár 10 je virtuálně replikován na rozměr [10, 10, 10].

Kód:

```
array([1, 2, 3]) * 10
```

↓ Logická reprezentace ↓

1	2	3	×	10	10	10
---	---	---	---	----	----	----

↓ Výsledek ↓

10	20	30
----	----	----

NumPy: Společný jazyk ekosystému

Univerzální rozhraní pro reprezentaci a manipulaci s daty.

□ Role v ekosystému:

- Společný datový kontejner pro obrázky, signály i tabulky.
- Standardizace předávání dat mezi C-rozšířeními.

□ Bohatá funkcionalita:

- `linalg`: Maticové operace a rozklady.
- `fft`: Transformace signálů v čase a frekvenci.
- `random`: Generátory náhodných distribucí.
- `ufuncs`: Rychlé matematické funkce (*sin*, *log*).

Dimenze	Typ dat v praxi
1D	Vektory, časové řady, zvuk
2D	Matice, tabulky, šedé obrázky
3D	Barevné obrázky, volumetrie

Slicing & Dimenze

Schopnost „krájet“ data bez nutnosti kopírování v paměti.

```
1  # 3D pole (např. RGB obrázků)
2  tensor = np.random.rand(100, 100, 3)
3
4  # Výřez: Všechny řádky a sloupce,
5  # ale pouze první barevný kanál (R)
6  channel_r = tensor[:, :, 0]
7
8  # Lineární algebra: Skalární součin
9  res = np.dot(matrix_a, matrix_b)
```

Interoperabilita

Knihovny jako Pandas, Scikit-learn nebo OpenCV „vidí“ pod kapotou stejný blok paměti ve formátu `ndarray`.

SciPy: Knihovna pro inženýrské výpočty

Nadstavba nad NumPy poskytující algoritmy pro vědu a techniku.

□ Klíčové moduly:

- `scipy.optimize`: Hledání kořenů modelů.
- `scipy.stats`: Rozdělení, popisná statistika.
- `scipy.signal`: Konvoluce, filtry, Fourier.
- `scipy.interpolate`: Prokládání dat (spliny).
- `scipy.integrate`: Numerická integrace.

□ Filozofie nástroje:

- Volá vysoce optimalizované rutiny v C a Fortranu (LAPACK, BLAS).
- Zaměřeno na numerickou stabilitu a přesnost.

NumPy vs. SciPy

NumPy je o ukládání a indexaci (pole).

SciPy je o tom, co s těmi poli děláte (výpočty).

Typický problém: Curve Fitting

Máte naměřená data a hledáte parametry funkce, která je nejlépe popisuje (proložení křivkou, regresní aproximace).

```
1  from scipy.optimize import curve_fit
2  import numpy as np
3
4  # Modelová funkce (např. lineární)
5  def model(x, a, b):
6      return a * x + b
7
8  x_data = np.array([0, 1, 2, 3])
9  y_data = np.array([1, 2.1, 3.9, 6.2])
10
11 # Nalezení optimálních parametrů a, b
12 params, _ = curve_fit(model, x_data, y_data)
13
14 print(f"Koeficienty: {params}")
15 # Výsledek: a ~ 1.7, b ~ 0.8
```

Pandas: Flexibilní datové struktury

Dává NumPy polím jména, typy a inteligentní indexaci.

□ Základní stavební bloky:

- **Series**: 1D pole s labely (indexem).
- **DataFrame**: 2D tabulka, kde každý sloupec je Series s vlastním datovým typem.

□ Data Alignment (Zarovnání):

- Operace neprobíhají podle pozice, ale podle **Indexu**.
- Při nesouladu indexů Pandas data nezahodí, ale automaticky vytvoří NaN.

w-stack-alignment.ipynb

```
1 import pandas as pd
2 # Tabulka A: Leden (100), Únor (200)
3 a = pd.Series([100, 200], index=['Jan', 'Feb'])
4 # Tabulka B: Únor (10), Březen (30)
5 b = pd.Series([10, 30], index=['Feb', 'Mar'])
6
7 print(a + b)
8 # Jan    NaN (chybí v B)
9 # Feb   210 (sečteno)
10 # Mar    NaN (chybí v A)
```

Klíčová výhoda

Heterogenní sloupce: V jedné tabulce kombinujete čísla, texty i časová razítka, aniž byste ztratili výkon NumPy jádra.

Pandas: Manipulace a SQL-like logika

Deklarativní přístup k transformaci surových dat na informace.

□ Hlavní přednosti:

- **I/O operace:** Snadné čtení z SQL, CSV, Excelu či Parquetu.
- **Agregace:** Komplexní pohledy na data přes groupby.
- **Časové řady:** Nativní podpora pro klouzavá okna, resampling a časová pásma.

□ Metoda .agg():

- Umožňuje aplikovat různé funkce na různé sloupce v jednom kroku.

SQL-like Workflow

Seskupení a agregace dat:

```
1 summary = (df[df['hodnota'] > 100]
2             .groupby('kategorie')
3             .agg({
4                 'hodnota': ['sum', 'mean'],
5                 'id': 'count'
6             }))
7
8 # Spojování jako v SQL
9 result = pd.merge(df1, df2, on='key')
```

Best Practice: Chaining

Díky tomu, že většina operací vrací nový DataFrame, můžete logiku řetězit za sebe. Kód je pak čitelný jako recept: „Vezmi data, vyfiltruj, seskup, spočítej.“

Matplotlib & Seaborn: Vizualizace

Klíčová vlastnost: Matplotlib poskytuje absolutní kontrolu, Seaborn přidává statistickou abstrakci.

- ❑ **Integrace:** Seaborn nativně rozumí Pandas DataFrame a automaticky mapuje sloupce na estetické prvky (barva, tvar).
- ❑ **Statistické vrstvy:** Automatické vykreslování regresních čar a distribucí.

————— Statistický graf —————

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Komplexní graf jedním příkazem
5 sns.set_theme(style="darkgrid")
6 tips = sns.load_dataset("tips")
7
8 sns.lmplot(data=tips, x="total_bill", y="tip", hue="smoker")
9 plt.title("Vztah útraty a spropitného")
10 plt.show()
```

Matplotlib funguje jako nízkoúrovňové vykreslovací jádro, zatímco Seaborn nabízí vysokoúrovňové rozhraní pro EDA. [w-stack-seaborn.ipynb](https://wstack-seaborn.ipynb)

Scikit-learn: Konzistentní API pro ML (strojové učení)

Klíčová vlastnost: Jednotné rozhraní `fit()` a `predict()` pro stovky algoritmů.

- **Pipelining:** Řetězení transformací (např. škálování) a modelu do jednoho celku.
- **Validace:** Robustní nástroje pro křížovou validaci a vyhodnocování metrik.

Workflow strojového učení

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.metrics import accuracy_score
4
5 # X = matice příznaků, y = vektor cílových hodnot
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
7
8 model = RandomForestClassifier()
9 model.fit(X_train, y_train)    # Učení modelu
10 preds = model.predict(X_test) # Predikce na nových datech
11
12 print(f"Accuracy: {accuracy_score(y_test, preds):.2f}")
```

Designový princip

Knihovna se soustředí na efektivitu a čitelnost. Pro hluboké učení (neuronové sítě) se pak typicky přechází na PyTorch nebo TensorFlow.

Tidy Data Architecture

Struktura určuje složitost kódu. Wide je pro lidi, Long je pro stroje.

□ Definice Tidy Data (Hadley Wickham):

- Každá **proměnná** má svůj sloupec.
- Každé **pozorování** má svůj řádek.
- Každá hodnota má svou buňku.

□ Proč chtít "Long" formát?

- **Škálovatelnost**: Přidání nového předmětu (např. "Chem") nezmění strukturu tabulky (nepřidá sloupec).
- **Sémantické mapování**: V Seabornu prostě napíšete `hue='Předmět'`. Ve Wide formátu musíte grafovat každý sloupec ručně.

Seaborn Hue (barva, odstín)

Unikátní technika pro vícerozměrnou analýzu dat. Umožňuje do 2D grafu přidat třetí dimenzi pomocí barevného kódování.

Transformace (`melt`)

Převod z *prezentace* (Wide) na *analýzu* (Long/Tidy).

Wide Format (Člověkem čitelný)

Jméno	Mat	Fyz
Adam	90	80
Eva	95	85

↓ Vektorizovaná operace `.melt()` ↓

Long / Tidy Format (Strojově zpracovatelný)

Jméno	Předmět	Body
Adam	Mat	90
Eva	Mat	95
Adam	Fyz	80
Eva	Fyz	85

[w-stack-tidy-data.ipynb](#)

View vs. Copy

Zápis do dat: Rozdíl mezi referencí a fyzickou alokací.

□ View (Pohled):

- Odkaz na část původní paměti.
- Změna ve View přepíše data v originálu.

□ Copy (Kopie):

- Úplně nový objekt v RAM.
- Vzniká při filtraci nebo přes `.copy()`.

Pravidlo: Přístup pomocí `.loc`

Používejte `df.loc[mask, 'col'] = val`.
Explicitně říkáte: „Najdi toto místo v původním objektu a změň ho.“ Vyhnete se varování `SettingWithCopyWarning`.

w-stack-copy-view.ipynb

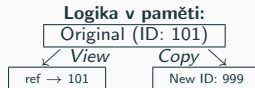
Problém: Chained Indexing

Špatně (vytváří varování):

```
1 # df[mask] může vrátit kopii!  
2 df[df['A'] > 5]['B'] = 10
```

Správně (explicitní zápis):

```
1 # Garantovaný zápis do originálu  
2 df.loc[df['A'] > 5, 'B'] = 10
```



Změna v 999 se do 101 nepromítne!

Nástupce? Pandas vs. Polars

Polars: Moderní alternativa postavená na Rustu a Apache Arrow.

- ❑ **Pandas (Legacy):**
Single-threaded, vysoká paměťová režie,
postaveno nad NumPy.
- ❑ **Polars (Modern):**
Multi-threaded (využívá všechna jádra), *Lazy Evaluation*, nulové kopírování dat.
- ❑ **Kdy zvolit co?**
Pandas pro malé datasety a široký ekosystém.
Polars pro GB+ data a maximální výkon.

Benchmark (Typicky)

Operace	Pandas	Polars
Join	1.0x	5–10x
Filter	1.0x	2–4x
Groupby	1.0x	8–12x

Polars automaticky optimalizuje dotazy před jejich spuštěním.

Klíčový rozdíl

Zatímco Pandas vykonává příkazy **okamžitě** (Eager), Polars v Lazy módu vytvoří **logický plán**, který před spuštěním optimalizuje. [w-stack-pandas-polars.ipynb](#)

Podle Úlohy (Doména)

- ❑ **NumPy**: Maticové operace, lineární algebra, tenzory.
- ❑ **Polars/Pandas**: Tabulky, SQL-like operace, Joiny.

Podle Škálování (RAM/CPU)

- ❑ **Pandas**: Malá data (<1GB), bohaté ekosystémové knihovny.
- ❑ **Polars**: Velká data (GB+), multi-core výkon, Lazy režim.

Best Practice

NumPy je pro **výpočetní jádra**, Polars je pro **datovou logistiku**. Pokud tabulka přesáhne jednotky GB, opusťte Pandas ve prospěch Polars.

Závěr: Data Science Toolbox v kostce

Klíčové principy pro efektivní práci s daty v Pythonu.

1. Hierarchie nástrojů

- ❑ **NumPy**: Základní výkon a paměťový layout (homogenní pole).
- ❑ **Pandas/Polars**: Logistika, tabulky a SQL operace nad NumPy/Arrow.
- ❑ **Seaborn**: Statistický vhled přes Tidy Data.

2. Architektura dat

- ❑ Preferujte **Long (Tidy)** formát pro vizualizaci a modelování.
- ❑ Vyhněte se řetězení (*Chained Indexing*), používejte `.loc`.

3. Výkonnostní mindset

- ❑ **Vektorizace > For-cykly**: Využívejte C-jádro knihoven.
- ❑ **Broadcasting**: Počítejte efektivně bez duplikace dat v RAM.
- ❑ **Lazy Evaluation**: U velkých dat (>1GB) uvažujte o Polars.

Pravidlo

Nástroj vybírejte podle **velikosti dat** a **typu operace**. Dobrý datový vědec nepíše jen funkční kód, ale kód, který respektuje limity paměti a CPU.